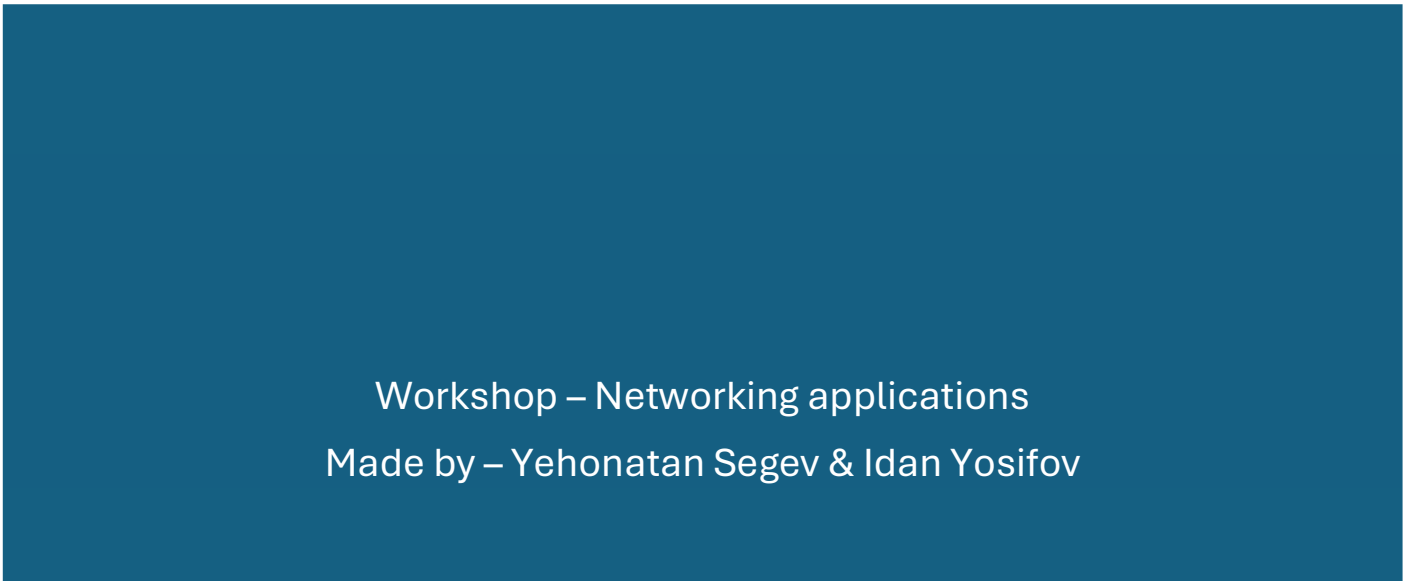




SIGNALLY



Workshop – Networking applications
Made by – Yehonatan Segev & Idan Yosifov

Table of Contents

Introduction	2
Background and Motivation.....	2
Project Goal	3
Specification	4
Target users	4
System features	4
User Flows/Scenarios	6
Non-functional requirements	8
External dependencies	9
Design.....	10
High-Level Architecture	10
Components.....	11
Deployment	12
Appendices	13-16

Introduction

Background and Motivation

Modern security and occupancy monitoring solutions often rely on sensors like cameras, motion detectors, or wearables. However, these approaches can be intrusive (violating privacy), expensive, or impractical in certain environments.

To address these limitations, Signally proposes a non-intrusive security approach based on the **existing Wi-Fi environment**. Wi-Fi networks already interact with smartphones and other personal devices, and these interactions can provide useful information about presence, movement, and device identity.

By combining connected-device detection, nearby-device probing, and advanced Wi-Fi sensing techniques such as **Channel State Information** (CSI), the system can better understand what is happening in the monitored space without relying on cameras or dedicated wearable sensors.

Our project, Signally, builds on this concept to provide a novel intrusion detection and occupancy monitoring system. It aims to detect the presence of a person or a smartphone device in each area using only Wi-Fi signals, without any cameras or dedicated sensors.

Project Goal

The goal of Signally is to develop a privacy-preserving Wi-Fi-based security system that can monitor a small environment, preprocess and analyze the information collected from it, and based on that information, infer meaningful occupancy information in real time. In particular, the system should be able to:

- **Detect and identify human presence (or a personal device)** in the monitored space (e.g., determine if someone is present or not).
- **Distinguish between an authorized user and a potential intruder** (friend or foe” detection), for example by recognizing known devices or patterns associated with household members.
- **Trigger real-time alerts** when unauthorized presence is detected
- **Operate with minimal user involvement**, providing an easy-to-use interface for monitoring and requiring little manual calibration or input from the user.

By achieving these objectives, Signally will offer a simple yet effective security tool that leverages Wi-Fi networks signals to sense occupancy. The next sections detail the expected functionality (Specification) and the internal design & architecture (Design) of the proposed solution.

Specification

Target users

The primary target users for this system are individuals and small organizations who need a lightweight indoor security or occupancy monitoring solution. Examples include: homeowners, apartment renters, small office owners, or any user interested in detecting intruders or monitoring presence in a private space **without deploying cameras or wearable devices**. The system is designed for **small-scale environments** (such as a single apartment, house, or small office floor), where a single Wi-Fi-based sensor device can cover the area of interest.

System features

The following are the main features of the proposed system.

Core Features (MVP):

1. **Wi-Fi-Based Environment Monitoring:** Continuously collect relevant information from the Wi-Fi environment, including connected devices, nearby-device indications, and signal-based sensing data.
2. **Device and User Authorization:** Maintain a list of known and unknown devices. A device is not considered authorized automatically; it must be approved by an administrator and assigned a role such as Admin, Family Member, or Guest.
3. **Correlation Engine:** Analyze the collected information together in order to understand the current situation. The engine combines indicators such as device status, user role, connection state, nearby presence, signal-based sensing, and the selected security mode to decide whether the environment is safe, requires review, or should trigger an alert.
4. **Dashboard for Monitoring:** Provide a user-facing dashboard that displays the current system state, detected devices or presence, recent events, and security alerts in near real time.

Optional Features:

1. **Activity Classification:** Determine the nature of the detected subject's activity or motion (for example, distinguishing between someone moving vs. standing still, or even specific postures). This can extend the system to basic activity recognition.
2. **Multi-Zone Support:** Support multiple sensing devices to cover different zones or rooms, allowing the system to monitor a larger area or multiple separate areas simultaneously. Identity recognition via gait signatures
3. **Identity Recognition:** Recognize specific individuals by their unique signal "signature" (for instance, we might use gait analysis or device-specific patterns in the Wi-Fi signal). This would enable the system to not only detect a presence but possibly identify who it is (among known users)

User Flows/Scenarios

To illustrate how the system will be used, here are a few key usage scenarios and workflows for Signally:

1) Scenario 1 – Real-Time monitoring:

- a) Admin opens the Signally monitoring interface to check status.
- b) The system displays the current state of the monitored area, including recent detections, known and unknown devices, and relevant alerts.
- c) As time progresses, the dashboard continues to update in real-time the status and logs recent events

2) Scenario 2 -New Authorized User Setup:

- a) A new user connects their personal device to the home Wi-Fi network.
- b) The system detects the new device and marks it as unknown or pending.
- c) The admin reviews the device and either approves it as an authorized device with a role, such as Family Member or Guest, or leaves it unauthorized.
- d) If the device is not approved within a configurable time window, the system may treat it as suspicious and escalate the event according to the current security mode.

3) Scenario 3 – Unknown Presence While Away Mode Is Active:

- a) The admin sets the system to Away mode before leaving the monitored area.
- b) While the system is armed, an unknown device or suspicious presence is detected.
- c) The correlation engine evaluates the event using available information such as device status, known users, nearby-device indications, and sensing data.
- d) If the event is classified as suspicious or unauthorized, the system triggers an alert and records the event in the activity log.

4) Scenario 4 – Nearby Device Detection:

- a) A phone or personal device is detected near the monitored area, even if it is not connected to the Wi-Fi network.
- b) The system treats this detection as an environmental signal and sends it to the correlation engine. The system analyzes this data to learn the “normal” environmental signature (baseline) when no human is present.
- c) If the system is in Home mode, the event may be logged or marked for review without an immediate alert.
- d) If the system is in Away mode and the device remains unknown after a configurable grace period, the system may escalate the event into an alert.

5) Scenario 5 – CSI-Based Presence and Baseline Learning:

- a) The system collects Wi-Fi signal measurements from the monitored environment.
 - b) During setup or calibration, the system can learn the normal signal behavior of the environment.
 - c) Future signal changes can be compared to this baseline to improve presence detection and reduce false positives.
 - d) This flow extends the system from device-based detection toward more advanced occupancy sensing.
- Refer to Appendices for further information/ visualization

Non-functional requirements

In addition to the functional features above, the system must meet several non-functional requirements to ensure it is practical and reliable:

- **Real-Time Performance:** The system should operate with low latency, i.e., it must process the incoming Wi-Fi data and update the dashboard or send alerts within a short delay (ideally a few seconds or less). Near-instant feedback is crucial for timely intrusion detection.
- **Privacy Preservation:** No sensitive audio or visual data is collected. The solution avoids cameras and wearable tracking, relying instead on Wi-Fi signals and network-related indicators.
- **Reliability and Stability:** The system should run continuously and be robust against crashes or disconnects. It must handle continuous data streaming and logging without significant data loss. The logging of events should be persistent so that no records of detections are lost over time.
- **Accuracy and False Alarms:** The detection algorithm should be reasonably accurate in distinguishing real presence from noise. The system should minimize false positives (false alerts when no intruder is actually present) and false negatives (failing to detect an actual intruder). Calibration and good algorithm design will be important to meet this requirement.
- **Scalability (within scope):** For a typical deployment in a home or small office, the system should handle the necessary load (for example, one or a few sensor devices streaming data).

External dependencies

The project relies on certain external hardware and software components:

- **Local Monitoring Component:** The system requires a local component inside the monitored environment, such as a router add-on, supported router integration, or a small Linux-based edge device. This component performs the local Wi-Fi/network monitoring while keeping the solution “deviceless” from the user’s perspective, since people do not need to wear or carry any dedicated sensor.
- **Wi-Fi Network Interface:** A Wi-Fi adapter is required for collecting Wi-Fi-related information, including connected-device data, nearby-device indications, and, in the extended version, CSI measurements
- **Network and Packet Processing Tools:** The system relies on networking tools and libraries, such as Scapy, to inspect network traffic and collect relevant Wi-Fi/network indicators.
- **CSI Extraction Tool/Software:** For the advanced sensing layer, specialized software may be required to capture Channel State Information from a compatible wireless interface.
- **Backend and Storage Platform:** The backend requires a web framework and a database layer for processing, storing, and exposing monitoring data.
- **User Interface Platform:** The front-end interface relies on mobile/web development frameworks for displaying the system state, events, and alerts to the u

Design

High-Level Architecture

The Signally system consists of the following major subsystems:

1. Local Monitoring and Back-End Architecture:

- Python with FastAPI is used to expose API endpoints, handle monitoring requests, and provide data to the application interface.
- Scapy and related networking tools are used to collect Wi-Fi/network indicators, such as connected devices, nearby-device indications, and packet-level information.
- A correlation engine analyzes the collected indicators together with the current security mode and authorization state, and decides whether the environment is safe, requires review, or should trigger an alert.
- The system uses a database layer through SQLAlchemy to store users, devices, roles, authorization status, and security events. In the current local deployment, SQLite is used, while a production or multi-user environment may use a stronger database such as PostgreSQL.....)

2. Front-End Architecture:

- A mobile application is built using React Native for cross-platform usage and responsive design.
- The interface displays the current security state, detected devices, authorization status, recent events, and alerts.

3. Future Scalable Architecture:

- In a larger deployment, a central server or cloud layer can manage multiple families, homes, routers, users, and permissions.
- This future architecture can support multi-tenant separation, allowing one system to serve several households while keeping each family's data logically isolated.

Components

The System main components:

- Raspberry Pi 5 that connects to the home network, preferably through Ethernet for stability.
- Micro USB Wi-Fi adapter that connects to the Raspberry Pi and collects Wi-Fi/network indications from nearby devices.
- Micro SD card to host a Linux OS and the system software.

Component	Role
Raspberry pi 4 / 5	Central processing unit for hosting the backend, monitoring logic, and correlation engine
Wi-Fi adapter	Collects Wi-Fi/network indications and supports future CSI extraction
Micro Sd	Hosts OS and software
Ethernet Cable & Power Supply	Provide Stable internet and power connections

- Refer to Appendices for further information/ visualization

Deployment

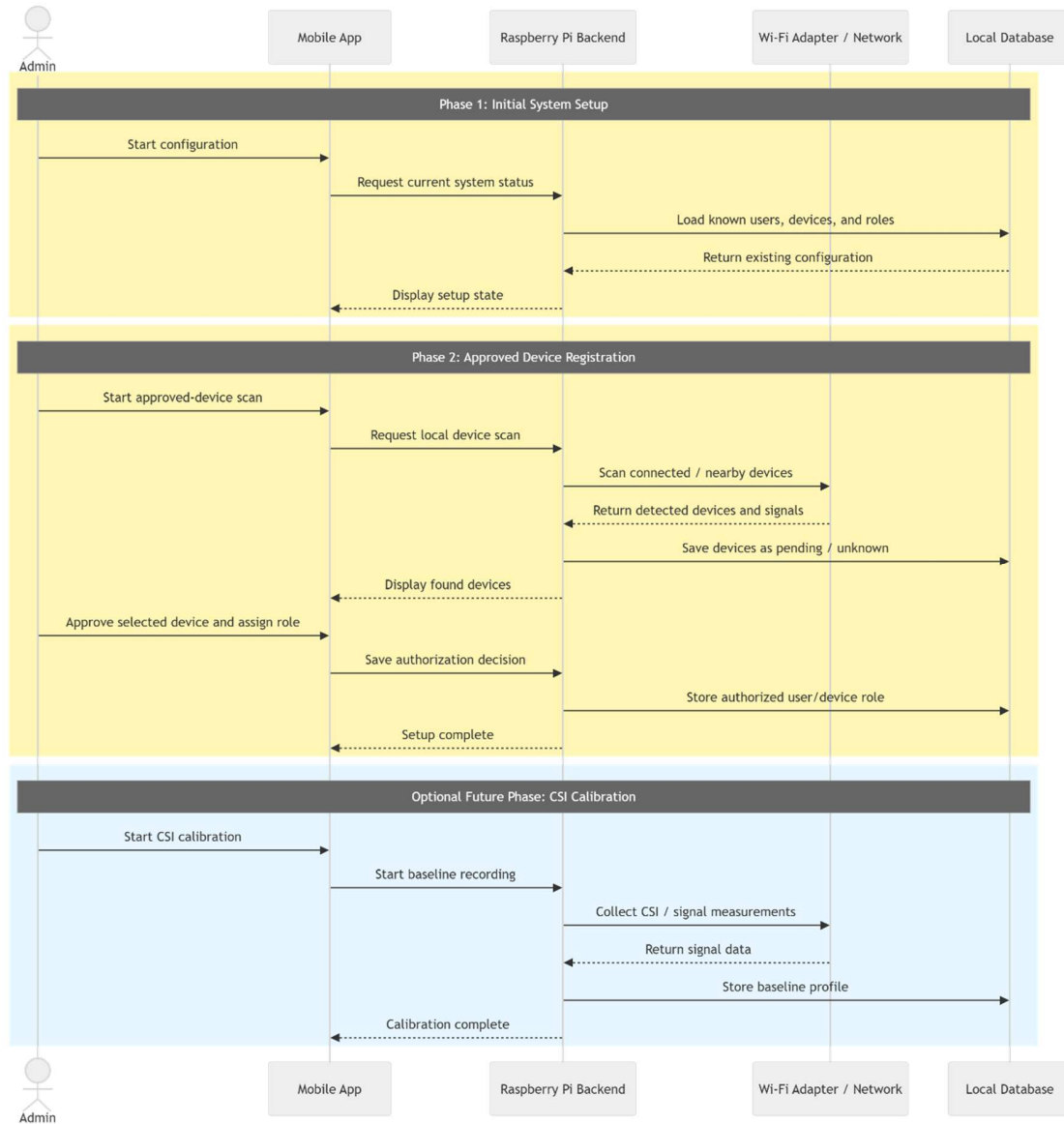
Means of deployment:

The product will run on a Raspberry Pi 5 connected to the home internet network, preferably via Ethernet for maximum stability. The Python / FastAPI backend will be deployed locally on the Pi as a background service. This local edge deployment allows Wi-Fi and network-related information to be collected, processed, and correlated close to the monitored environment, while storing relevant users, devices, authorization states, and security events in a local database.

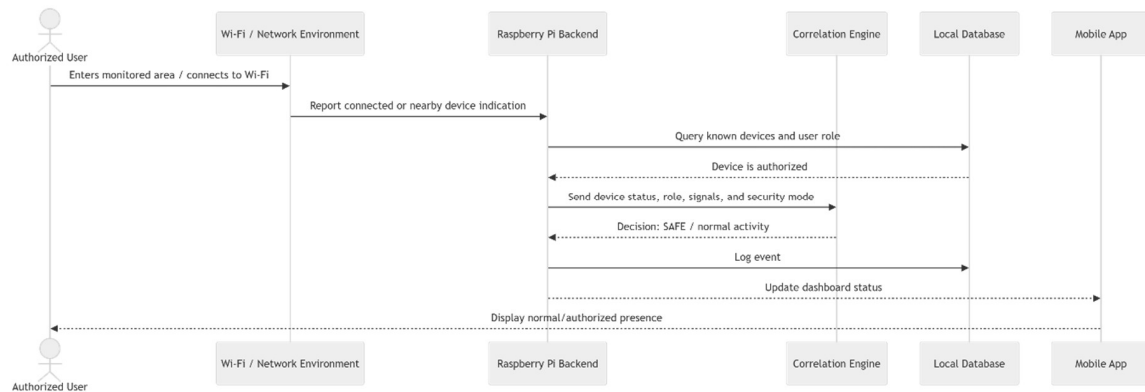
In future versions, the local deployment may be extended with a central server or cloud layer to support multiple homes, routers, users, and permission models.

Appendices

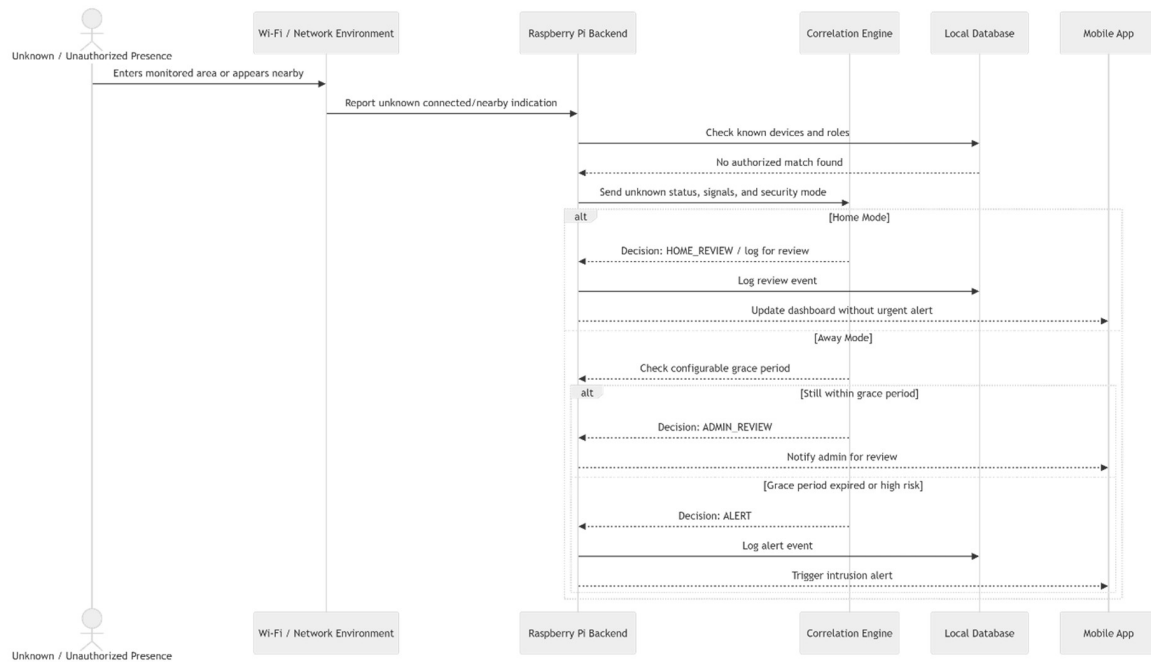
Configuration / Setup Sequence Diagram-



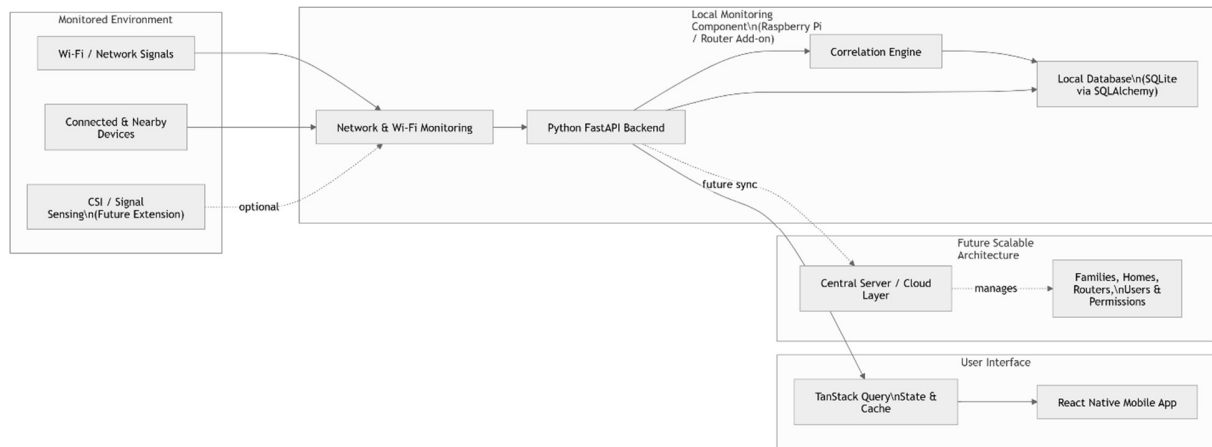
Authorized/Friendly User Sequence Diagram-



Intruder Sequence Diagram-



Architecture sketch-



Wireframe sketch-

